Beyond the Basics

Pseudocode & Python · Flowcharts · Logic Gates

A subtopic-by-subtopic study guide for everything **after** the basics. Every subtopic follows the same recipe: an explanation built on analogies and worked through with real values, examples in pseudocode, Python and flowchart form, the common errors, and practice questions — with every answer collected at the back of the guide in full detail.

Based on the **Cambridge O Level Computer Science 2210** syllabus for 2026–2028, Paper 2: Algorithms, Programming and Logic (Topics 7–10), including the 15-mark scenario question.

Compiled 12 November 2026

Contents

How this guide works 4

1. The big picture — the analogy map 4

2. Iteration and nested constructs 5

2.1 FOR loops — counting laps on the track . . . 5

2.2 WHILE loops — the pizza check . . . 6

2.3 REPEAT ... UNTIL — stirring the tea . . . 7

2.4 Totalling and counting — the till and the clicker . . . 8

2.5 Nested IF and nested loops — triage and the cinema . . . 9

3. Strings 11

3.1 String operations — the bead necklace . . . 11

3.2 Character by character — sliding the magnifying glass . . . 12

4. Arrays and data tables 14

4.1 One-dimensional arrays — the corridor of lockers . . . 14

4.2 Two-dimensional arrays — the cinema . . . 15

4.3 Database tables and SQL — the spreadsheet with rules . . . 16

5. Procedures and functions 18

5.1 Procedures — the recipe card . . . 18

5.2 Functions — the blender . . . 19

5.3 Local and global — notepad vs whiteboard . . . 20

6. Files 21

6.1 File operations — the filing cabinet . . . 21

7. Standard algorithms and trace tables 22

7.1 Maximum, minimum and average — the tallest kid . . . 22

7.2 Linear search — the register scan . . . 24

7.3 Bubble sort — the fizzy drink . . . 25

7.4 Trace tables — the flight recorder . . . 26

8. Program development and testing 27

8.1 The PDLC and validation checks — the bouncer . . . 27

8.2 Verification and test data — the proofreader's checklist . . . 28

9. Flowcharts 29

9.1 Symbols and conversion — the road map . . . 29

10. Boolean logic and logic gates 30

10.1 The seven gates — tiny decision machines . . . 30

10.2 Conversions — statement, expression, circuit, truth table 31

11. The scenario question **32**

Answers **33**

Section 2 — Iteration and nested constructs 33

Section 3 — Strings 36

Section 4 — Arrays and data tables 37

Section 5 — Procedures and functions 39

Section 6 — Files 40

Section 7 — Standard algorithms and trace tables 41

Section 8 — Program development and testing 43

Section 9 — Flowcharts 44

Section 10 — Boolean logic and logic gates 44

Section 11 — The scenario question 45

Appendix A — pseudocode and Python side-by-side reference **47**

Appendix B — command words **48**

How this guide works

Every subtopic in this guide follows the same recipe:

1. **Explanation** — the idea through an analogy, then worked through with **real values** (actual numbers you can check on paper).
2. **Examples** — the same algorithm in **pseudocode**, **Python** (shortcuts used openly), and **flowchart** form.
3. **Common errors** — the traps for that subtopic, at the moment you meet them.
4. **Practice questions** — try them unaided. **All answers are collected at the back of the guide** (section 12) with full worked explanations and mark points.

> **Note**
> **The golden exam rule.** In the real Paper 2 exam, every coding answer must be written in **Cambridge pseudocode** — answers in Python score zero, **except** in the final 15-mark scenario question, where pseudocode **or** Python/Visual Basic/Java is allowed. Syntax is never marked: the **logic** earns the marks. Operators must be precise: write `x > y`, never "x is greater than y".

Two things about this guide's layout:

- **Python shortcuts** (elif, +=, for-each, f-strings...) appear inside the examples where they belong, always after the fundamental form they replace.
- **Databases and SQL are taught inside the arrays section (4.3).** A database table *is* parallel arrays of records, and an SQL query is just a filter loop over them — the same mental model. (The syllabus lists databases as its own Topic 9, so it is examined separately, but you learn it here because that is what it *is*. If you understand arrays, you already understand SQL.)

1. The big picture — the analogy map

Every idea in this guide has a mental model. Keep this table; when something feels slippery, find its row and rebuild from the analogy.

Idea	Analogy	Real values
Variable	labelled box	`Total ← 0`, then `Total ← Total + 5` holds 5
FOR loop	counting laps	5 laps: I takes 1, 2, 3, 4, 5
WHILE loop	eating pizza	eat while `Slices > 0`
REPEAT UNTIL	stirring tea	stir until sugar dissolved
Accumulator	till receipt	2 + 5 + 1 = 8
Counter	clicker	3 people in → Count = 3
1D array	numbered lockers	`A = [7, 2, 9, 4]`; `A[3] = 9`
2D array	cinema seats	`Grid[2, 3]` = row 2, seat 3
String	bead necklace	`"HELLO"`: LENGTH = 5
Procedure	recipe card	`CALL PrintLine('*', 5)` prints *
Function	blender	`Square(3)` hands back 9
Local vs global	notepad vs whiteboard	local `Total = 10` leaves global `Total = 5`
File	filing cabinet	`Scores.txt` survives the program
Linear search	register scan	find 9 in `[7, 2, 9, 4]` → position 3
Bubble sort	fizzy drink	`[5, 3, 4]` → `[3, 4, 5]`

Idea	Analogy	Real values
Trace table	flight recorder	record every variable at every step
Validation	club bouncer	age 13–19 rejects 20, accepts 15
Verification	proofreader	type the password twice, compare
Logic gates	tiny decision machines	two-key safe: A AND B
Database table	spreadsheet	StudentID = exam candidate number

2. Iteration and nested constructs

2.1 FOR loops — counting laps on the track

Explanation. A FOR loop is a planned number of laps: you decide "5 laps" **before** you start, and a lap counter (`I`) ticks 1, 2, 3, 4, 5. With `FOR I ← 1 TO 5`, the body runs exactly 5 times with `I` holding 1, then 2, then 3, 4, 5. If the track is closed before you start (`FOR I ← 1 TO 0`), you run **zero** laps. A step is the lap size: `STEP 2` on 1 to 9 visits 1, 3, 5, 7, 9 — five visits, not nine.

Examples. Output the lap numbers 1 to 5:

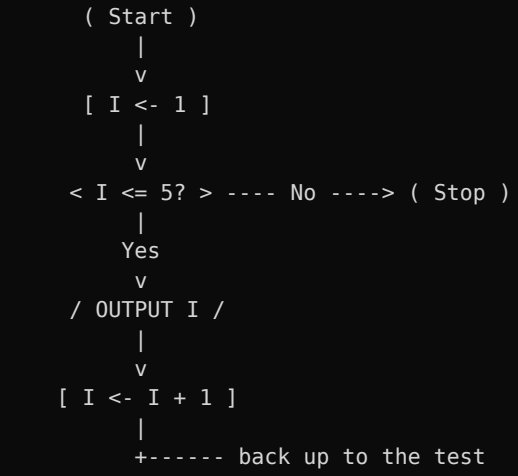
Pseudocode:

```
FOR I ← 1 TO 5
    OUTPUT I
NEXT I
```

Python (with shortcut):

```
for i in range(1, 6):      # range stops BEFORE 6, so laps 1..5
    print(i)
```

Flowchart:



Common errors

Fence posts. `TO 5` means **5** laps (1, 2, 3, 4, 5) — not 4. Python's `range(1, 5)` gives only 1 to 4; to include 5 write `range(1, 6)`. When a loop "prints one too few", count the fence posts first. Also: if start > end the body never runs — that is correct behaviour, not a bug.

Practice questions

Q1. **[3 marks]**

Write an algorithm that calculates the sum of the squares of the numbers 1 to 10 (1 + 4 + 9 + ... + 100) and outputs the total, in pseudocode **and** Python. The value 385 must not appear as a literal.

Q2. **[2 marks]**

State exactly what this pseudocode outputs, and give the Python line with the same effect.

```
FOR I ← 3 TO 7 STEP 2
    OUTPUT I
NEXT I
```

2.2 WHILE loops — the pizza check

Explanation. A WHILE loop asks **before every** slice: "is there pizza left?" Start with 3 slices: check 3 > 0 (eat → 2), check 2 > 0 (eat → 1), check 1 > 0 (eat → 0), check 0 > 0 (stop). Three slices eaten. Start with 0 slices and you eat **none** — the body can run zero times. The condition is re-tested before each run, and the loop lives only as long as it stays true.

Examples. Keep asking for the password until it is correct:

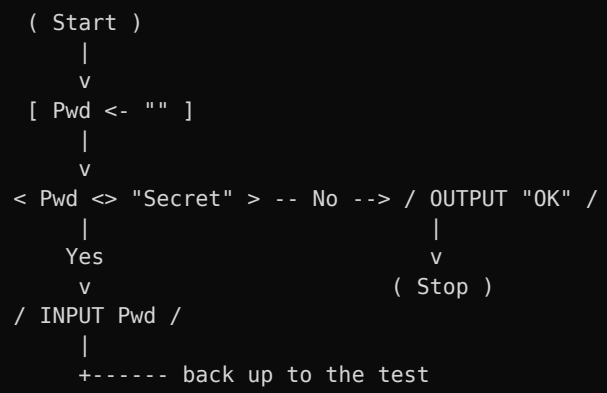
Pseudocode:

```
Password ← ""
WHILE Password <> "Secret" DO
    INPUT Password
ENDWHILE
OUTPUT "Access granted"
```

Python (with shortcut):

```
password = ""
while password != "Secret":
    password = input("Password: ")
print("Access granted")
```

Flowchart:



Common errors

The hamster wheel. A WHILE loop whose counter is never incremented runs forever:

```
I ← 1
WHILE I <= 5 DO
    OUTPUT I
    // forgot: I ← I + 1
ENDWHILE
```

FOR/NEXT increments for you; in a WHILE loop **you** must write the increment. Also mind the comparison: `I < 5` prints 1 to 4; `I <= 5` prints 1 to 5.

Practice questions

Q3. **[3 marks]**

Rewrite this FOR loop as a WHILE loop in pseudocode **and** Python. It must produce exactly the same output.

```
FOR I ← 1 TO 10
    OUTPUT I * 2
NEXT I
```

Q4. **[2 marks]**

This loop should output 1 to 5 but outputs 1 to 4. Identify the error and write the corrected line (the same bug exists in the Python version — fix it there too).

```
I ← 1
WHILE I < 5 DO
    OUTPUT I
    I ← I + 1
ENDWHILE
```

2.3 REPEAT ... UNTIL — stirring the tea

Explanation. REPEAT ... UNTIL is the stir-and-check loop: you **always stir at least once**, then check "dissolved yet?" — and the loop **stops when the answer is TRUE**. Asking for a password: attempt "abc" (not "Secret" — stir again), attempt "xyz" (again), attempt "Secret" (stop). Three attempts, and any number of wrong ones is allowed. The test happens **after** the body — that is why it always runs at least once, and why the UNTIL condition is the **stopping** state, the opposite of Python's `while`.

Examples. Ask for the password until it is correct (at least one attempt guaranteed):

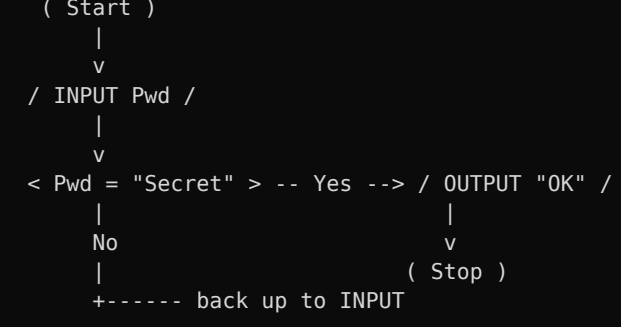
Pseudocode:

```
REPEAT
    INPUT Password
UNTIL Password = "Secret"          // stops when TRUE
OUTPUT "Access granted"
```

Python (with shortcut):

```
while True:
    password = input("Password: ")
    if password == "Secret":
        break                       # the UNTIL test
print("Access granted")
```

Flowchart:



Common errors

UNTIL polarity. `UNTIL` stops when the condition is **TRUE** — the sugar must be dissolved. Writing the wrong side inverts the loop:

```
REPEAT
    INPUT Password
UNTIL Password <> "Secret"    // WRONG: stops when password is NOT "Secret"
```

Write the state you want to **stop** on: `UNTIL Password = "Secret"`. The Python twin mistake is forgetting the `if condition: break` entirely.

Practice questions

Q5. **[3 marks]**

This pseudocode is supposed to keep asking until the user enters a valid mark between 0 and 100 inclusive. It does the opposite. Identify the error and fix it.

```
REPEAT
    INPUT Mark
UNTIL Mark < 0 OR Mark > 100
```

Q6. **[2 marks]**

For each situation, state whether FOR, WHILE or REPEAT … UNTIL fits best and justify in one sentence: (a) input exactly 20 exam marks; (b) keep asking for a password until it is "Open123"; (c) keep offering pizza slices while the box is not empty.

2.4 Totalling and counting — the till and the clicker

Explanation. An accumulator is the **till receipt**: scan prices 2, 5 and 1 and the display goes 0 → 2 → 7 → 8. A counter is the **clicker**: clicking once for each value over 4 among 2, 5, 1 gives one click (only 5 qualifies). Both must be **set to zero before the loop** — the receipt starts at 0.00 — and each is updated inside the loop, one step at a time.

Examples. Input 5 prices, output the total and how many were over 4:

Pseudocode:

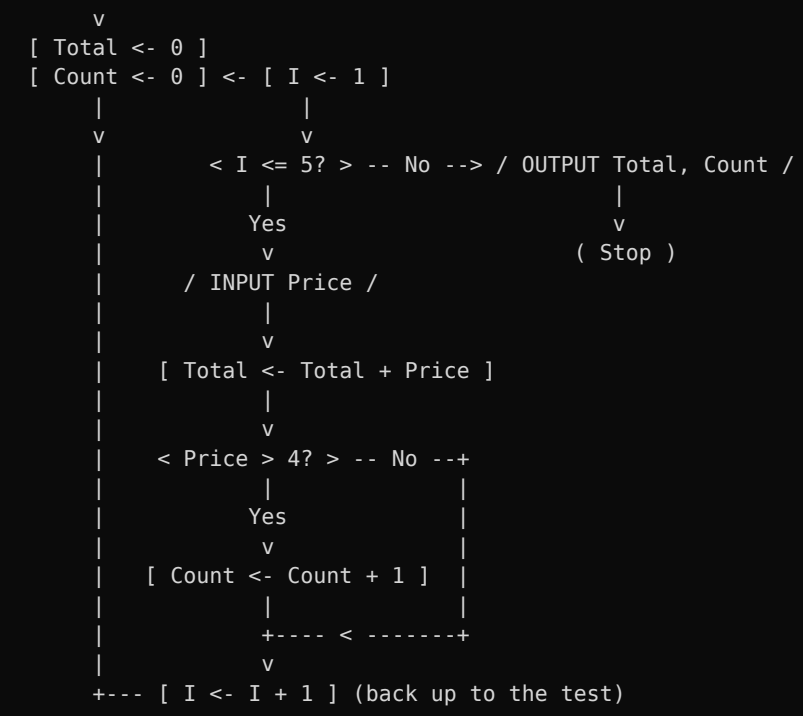
```
Total ← 0
Count ← 0
FOR I ← 1 TO 5
    INPUT Price
    Total ← Total + Price
    IF Price > 4
        THEN
            Count ← Count + 1
    ENDIF
NEXT I
OUTPUT Total, Count
```

Python (with shortcuts):

```
total = 0
count = 0
for i in range(1, 6):
    price = float(input("Price: "))
    total += price                  # shortcut for total = total + price
    if price > 4:
        count += 1
print(total, count)
```

Flowchart:

```
  ( Start )
      |
```



Common errors

The till must start at zero. `Total ← Total + Value` without `Total ← 0` first adds to a garbage starting value. And keep the pair apart: prices go in the `Total`, clicks go in the `Count` — `Count ← Count + Price` runs perfectly and is perfectly wrong.

Practice questions

Q7. **[3 marks]**

This pseudocode should output the sum 1 + 2 + 3 + 4 + 5 (= 15). Identify the **two** errors and write the corrected code.

```
FOR I ← 1 TO 4
    Total ← Total + I
NEXT I
OUTPUT Total
```

Q8. **[3 marks]**

Write an algorithm in pseudocode **and** Python that inputs 5 integers and outputs how many of them are greater than 10.

2.5 Nested IF and nested loops — triage and the cinema

Explanation. A nested IF is a **triage of questions**: the second question is only asked if the first sent you down a particular path. Mark 72: is it ≥ 80? No → is it ≥ 60? Yes → "Grade B". One question answered, the next only where needed. Nested loops are the **cinema**: walk every seat in every row — for a 2×3 grid holding [[3, 1, 4], [2, 5, 6]], the row totals are 3+1+4 = 8 and 2+5+6 = 13. The syllabus allows at most **three levels** of nesting.

Examples. Grade bands (score 72 → "Grade B") and a row-total loop:

Pseudocode:

```
IF Score >= 80
    THEN
        OUTPUT "Grade A"
    ELSE
        IF Score >= 60
```

```
            THEN
                OUTPUT "Grade B"
            ELSE
                OUTPUT "Grade C"
        ENDIF
ENDIF
```

```
FOR Row ← 1 TO 2
Sum ← 0
FOR Col ← 1 TO 3
Sum ← Sum + Grid[Row, Col]
NEXT Col
OUTPUT "Row ", Row, " total is ", Sum
NEXT Row
```

Python (nested if; shortcut version after):

```
if score >= 80:
print("Grade A")
else:
if score >= 60:
print("Grade B")
else:
print("Grade C")
```

```
for row in range(1, 3):
total = 0
for col in range(1, 4):
total += grid[row - 1][col - 1]
print("Row", row, "total is", total)
```

Python shortcut — the elif ladder (same logic, flatter; pseudocode has no elif, so exams need the nested fo

```
if score >= 80:
print("Grade A")
elif score >= 60:
print("Grade B")
else:
print("Grade C")
```

Flowchart (nested IF):

```
( Start )
|
v
< Score >= 80? > -- Yes --> / OUTPUT "A" / --> ( Stop )
|
No
v
< Score >= 60? > -- Yes --> / OUTPUT "B" / --> ( Stop )
|
No
v
/ OUTPUT "C" /
|
v
( Stop )
```

Common errors

Pairs and indentation. Every nested `IF` needs its **own** `ENDIF` (two IFs above, two ENDIFs below). In Python the nested IF belongs **inside** the else block — one indent deeper; a mis-indented IF silently joins the wrong branch. And in nested loops, use the **right counter** in the body: `Grid[Row, Row]` looks up the wrong seat.

Practice questions

Q9. **[3 marks]**

Write nested IF statements in pseudocode **and** Python that output "Child" if Age < 13, "Teen" if Age < 20 (but ≥ 13), and "Adult" otherwise. State what your code prints for ages 10, 15 and 22.

Q10. **[3 marks]**

Rewrite this Python program using three shortcuts: for-each, += and elif.

```
count = 0
for i in range(0, len(marks)):
    mark = marks[i]
    if mark >= 80:
        count = count + 1
        print("Distinction")
    else:
        if mark >= 50:
            count = count + 1
            print("Pass")
        else:
            print("Fail")
print(count)
```

Q11. **[2 marks]**

State exactly what this Python code prints, and give the pseudocode equivalent of the loop.

```
n = 5
total = 0
while n > 0:
    total += n
    n -= 1
print(total, n)
```

3. Strings

3.1 String operations — the bead necklace

Explanation. A string is a necklace of beads, and the operations work exactly as the analogy suggests. Take `S = "HELLO"`: `LENGTH(S)` counts 5 beads. `SUBSTRING(S, 2, 3)` snips 3 beads starting at bead 2 — "ELL". `UCASE("ell")` repaints to "ELL"; `LCASE('W')` gives 'w'. Strings join with +: `"Good" + " " + "morning"` is "Good morning". Cambridge numbers beads from **1**; Python from **0** — bead 2 for Cambridge is index 1 in Python.

Examples. From `Text = "Computer"`, snip the 3 characters starting at position 3 ("mpu") and upper-case the first letter:

Pseudocode:

```
Text ← "Computer"
Part ← SUBSTRING(Text, 3, 3)
First ← UCASE(SUBSTRING(Text, 1, 1))
OUTPUT Part, First          // outputs: mpu  C
```

Python (with shortcuts):

```
text = "Computer"
part = text[2:5]              # offset: position 3 = index 2
first = text[0:1].upper()
print(part, first)            # outputs: mpu C
```

Flowchart:

```
 ( Start )
     |
     v
 [ Text <- "Computer" ]
     |
     v
 [ Part <- SUBSTRING(Text, 3, 3) ]
     |
     v
 [ First <- UCASE(SUBSTRING(Text, 1, 1)) ]
     |
     v
 / OUTPUT Part, First /
     |
     v
 ( Stop )
```

Common errors

The offset trap. Cambridge bead 3 is Python bead 2: `SUBSTRING(S, 3, 3)` = `s[2:5]` — the rule is `s[p-1 : p-1+l]`. When a substring answer returns "the wrong letters", check the offset before anything else.

Practice questions

Q12. **[3 marks]**

The variable Text contains "Computer". Evaluate (a) `LENGTH(Text)` (b) `SUBSTRING(Text, 3, 3)` (c) `UCASE(SUBSTRING(Text, 2, 1))` and give the Python equivalent of each expression.

Q13. **[2 marks]**

Evaluate `SUBSTRING("HELLO", 2, 3)` and `LENGTH(SUBSTRING("HELLO", 2, 3))`. Then give the Python slice that produces the same substring.

3.2 Character by character — sliding the magnifying glass

Explanation. Processing a string bead by bead means a loop from 1 to `LENGTH(s)`, looking at one bead each time. Count the vowels in "Computer": C no, **o** yes (1), m no, p no, **u** yes (2), t no, **e** yes (3), r no — **3**. The loop visits 8 beads, the counter clicks 3 times. This one pattern answers most string questions: counting, searching, checking rules, rebuilding (reversing = the same walk in **reverse**, bead 8 down to 1, stitching as you go).

Examples. Count vowels (case-insensitive) and reverse the string:

Pseudocode:

```
INPUT S
Count ← 0
FOR I ← 1 TO LENGTH(S)
    Ch ← UCASE(SUBSTRING(S, I, 1))
    IF Ch = "A" OR Ch = "E" OR Ch = "I" OR Ch = "O" OR Ch = "U"
        THEN
            Count ← Count + 1
    ENDIF
NEXT I
OUTPUT Count
```

```
Reversed ← ""
FOR I ← LENGTH(S) TO 1 STEP -1
Reversed ← Reversed + SUBSTRING(S, I, 1)
NEXT I
OUTPUT Reversed
```

Python (with shortcuts):

```
s = input("Text: ")
count = 0
for ch in s: # for-each shortcut
if ch.upper() in "AEIOU": # 'in' retires the OR chain
count += 1
print(count)
```

```
reversed_text = ""
for i in range(len(s) - 1, -1, -1):
reversed_text += s[i]
print(reversed_text)
print(s[::-1]) # slicing shortcut: same result
print("-" * 30) # repetition: 30 dashes
print(f"Count: {count}") # f-string shortcut
```

Flowchart (vowel counter):

```
( Start )
|
v
/ INPUT S / [ Count <- 0 ] [ I <- 1 ]
| | |
+--------------+--------------+
v
< I <= LENGTH(S)? > -- No --> / OUTPUT Count /
| |
Yes v
v ( Stop )
[ Ch <- UCASE(SUBSTRING(S, I, 1)) ]
|
v
< Ch = "A" OR "E" OR "I" OR "O" OR "U"? >
| No |
| Yes
| v
| [ Count <- Count + 1 ]
| |
+-------- < ---------+
v
[ I <- I + 1 ] (back up to the test)
```

Common errors

Three string traps. (1) **Case sensitivity:** input "secret" fails against = `"Secret"` — normalise with `UCASE` / `.upper()` when the question wants case-insensitive matching. (2) **Text + number in print:** `print("Total: " + total)` is a TypeError — use `print("Total:", total)`, `str(total)`, or an f-string. (3) **Slicing past the end:** `s[0:100]` is harmless in Python (it just stops), but `SUBSTRING` past the end in pseudocode is a logic error — check lengths first.

Practice questions

Q14. **[4 marks]**

Write an algorithm in pseudocode **and** Python that inputs a string S and counts how many times the letter "e" appears, in either case. Output the count.

Q15. **[3 marks]**

Write an algorithm in pseudocode **and** Python that inputs S and outputs it reversed. For "ABC" the output must be "CBA".

Q16. **[3 marks]**

Write one line of Python for each task using a shortcut: (a) print 30 dashes; (b) test whether ch is a vowel using `in`; (c) output s reversed without a loop.

Q17. **[2 marks]**

This Python code is run. Write the exact output, then the pseudocode version of the same program.

```
s = "Computer"
print(len(s), s[2:5], s.upper())
```

4. Arrays and data tables

4.1 One-dimensional arrays — the corridor of lockers

Explanation. An array is a corridor of identical, numbered lockers. With `Marks = [50, 70, 90, 60]` in lockers 1 to 4: `Marks[3]` is 90. Cambridge corridors number from 1; Python's from 0 — the same 90 is `marks[2]`. Declare the corridor with a fixed size (`ARRAY[1:4] OF INTEGER`), then walk it with a loop: locker `I` for `I` = 1, 2, 3, 4. Asking for `Marks[5]` in a corridor of 4 is a locker that doesn't exist (**index out of range**). In Python the list can also grow: `marks.append(75)` adds a new locker at the end.

Examples. Declare 4 marks, fill from input, echo back locker 3:

Pseudocode:

```
DECLARE Marks : ARRAY[1:4] OF INTEGER
FOR I ← 1 TO 4
    INPUT Marks[I]
NEXT I
OUTPUT Marks[3]
```

Python (with shortcuts):

```
marks = []
for i in range(0, 4):
    marks.append(int(input("Mark: ")))
print(marks[2])                  # locker 3 = index 2
```

for mark in marks: # for-each: walk without indexes
print(mark)

```
**Flowchart:**
```

(Start)
|
v
[I <- 1]
|
v
< I <= 4? > -- No --> / OUTPUT Marks[3] / --> (Stop)
|

Yes
v
/ INPUT Marks[I] /
|
v
[I <- I + 1] (back up to the test)

Common errors

Lockers that don't exist. For `ARRAY[1:30]` the lockers are `A[1]` to `A[30]`: a loop from 0, or to 31, walks past the corridor's end. And the numbering differs between languages — Cambridge `Marks[5]` is Python `marks[4]` (the same offset trap as strings).

Practice questions

Q18. **[3 marks]**

Write an algorithm in pseudocode **and** Python to declare an array of 20 integer exam marks and fill it with values typed by the user.

Q19. **[4 marks]**

The array A holds 2, 7, 4, 9 in positions 1 to 4. Trace the algorithm below (columns `I`, `A[I]`, `Max`), and state what is output.

```
Max ← A[1]
FOR I ← 2 TO 4
    IF A[I] > Max
        THEN
            Max ← A[I]
    ENDIF
NEXT I
OUTPUT Max
```

Q20. **[4 marks]**

This Python program is meant to total the scores 50, 70, 90 and print the total, but it crashes. Identify the **two** errors and write the corrected program.

```
scores = [50, 70, 90]
total = 0
for i in range(1, 4):
    total = total + scores[i]
print("Total: " + total)
```

4.2 Two-dimensional arrays — the cinema

Explanation. A 2D array is a cinema: rows of seats, `Grid[Row, Col]`. For the 2×3 cinema [[3, 1, 4], [2, 5, 6]] — row 1 seats hold 3, 1, 4; row 2 holds 2, 5, 6 — `Grid[2, 3]` is 6 and the row totals are 3+1+4 = **8** and 2+5+6 = **13**. Walking the whole cinema needs **nested loops**: the outer loop takes a row, the inner loop sweeps every seat in it.

Examples. Output each row's total:

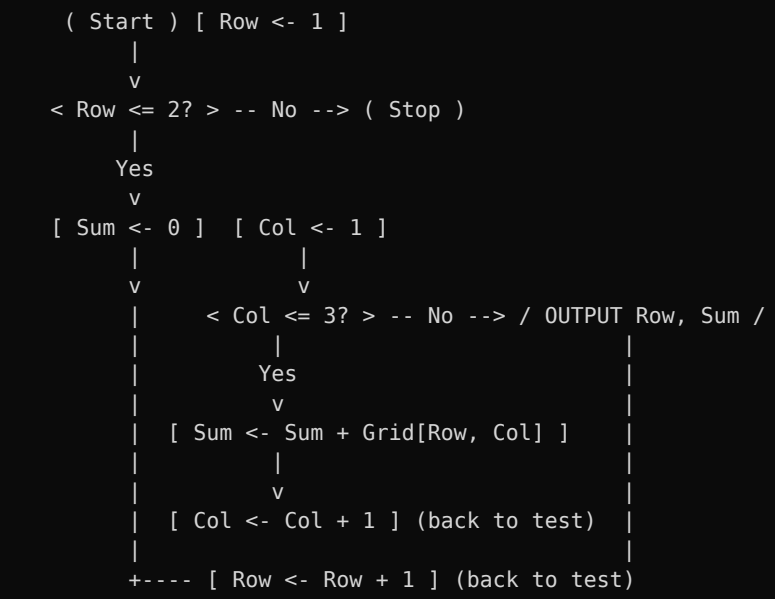
Pseudocode:

```
FOR Row ← 1 TO 2
    Sum ← 0
    FOR Col ← 1 TO 3
        Sum ← Sum + Grid[Row, Col]
    NEXT Col
    OUTPUT "Row ", Row, " total is ", Sum
NEXT Row
```

Python (with shortcuts):

```
for row in range(0, 2):
    total = 0
    for col in range(0, 3):
        total += grid[row][col]
    print("Row", row + 1, "total is", total)
```

Flowchart:



Common errors
Wrong index in the body. `Grid[Row, Row]` (using the outer counter twice) silently reads the wrong seat — always check that the inner loop's counter is the one used for the column. Also mind the bases: pseudocode `Grid[2, 3]` is Python `grid[1][2]`.

Practice questions

Q21. **[4 marks]**

A 2D array `Grid` has 2 rows and 3 columns: row 1 is 3, 1, 4 and row 2 is 2, 5, 6. Write an algorithm (pseudocode **and** Python) to output the total of each row. It must work for any stored values.

Q22. **[2 marks]**

Show how to build a list/array `Squares` holding 1, 4, 9, 16, 25 by looping — in both languages.

4.3 Database tables and SQL — the spreadsheet with rules

Explanation. A database table is a spreadsheet — and a spreadsheet is just **parallel arrays plus rules**. Each **record** is a row (one student), each **field** a column (Name, Mark); the **primary key** is the exam candidate number: unique, never blank, never reused. **SQL** asks the spreadsheet questions in one line. Look at this table as arrays: `name = ["Aisha","Ben","Chen","Dina","Elias"]`, `class_ = ["9A","9B","9A","9B","9A"]`, `mark = [84, 72, 91, 65, 78]` — and the query *"9A students with mark ≥ 80, highest first"* is just a **filter loop** over those arrays (plus a sort). The syllabus examines databases as Topic 9, but the thinking is pure arrays — which is why it lives here.

Examples. The same question in SQL, pseudocode and Python:

StudentID	Name	Class	Mark
101	Aisha	9A	84
102	Ben	9B	72

StudentID	Name	Class	Mark
103	Chen	9A	91
104	Dina	9B	65
105	Elias	9A	78

SQL (the examinable keywords: SELECT, FROM, WHERE, ORDER BY, SUM, COUNT, AND, OR):

```
SELECT Name FROM Students
WHERE Class = "9A" AND Mark >= 80
ORDER BY Mark DESCENDING
```

Output: Chen (91), Aisha (84).

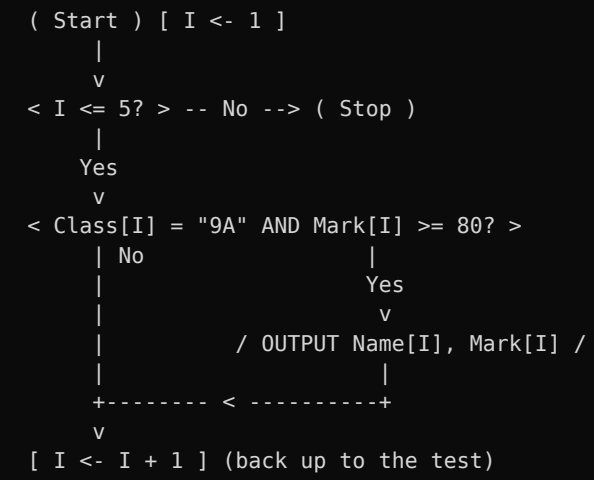
Pseudocode (the same filter as a loop):

```
FOR I ← 1 TO 5
    IF Class[I] = "9A" AND Mark[I] >= 80
        THEN
            OUTPUT Name[I], Mark[I]
    ENDIF
NEXT I
```

Python (with shortcuts):

```
for i in range(0, 5):
    if class_[i] == "9A" and mark[i] >= 80:
        print(name[i], mark[i])
```

Flowchart (the query as a road):



Common errors

Both conditions, every row. When predicting SQL output, check each record against **all** the WHERE conditions — AND means both must hold (in the table above, Ben has 72 ≥ 80 but is 9B, so he is out). And choose primary keys for uniqueness: `Name` can repeat; `StudentID` cannot.

Practice questions

Q23. **[3 marks]**

Write an SQL query to list the `Name` of every student in class "9A" with a `Mark` of 80 or more, highest mark first.

Q24. **[3 marks]**

State the exact output of:

```
SELECT COUNT(StudentID) FROM Students WHERE Mark >= 70 AND Class = "9A"
```

Q25. **[2 marks]**

A school table has fields `Name`, `DateOfBirth`, `Class` and `StudentID`. Suggest a suitable primary key and justify your choice.

Q26. **[3 marks]**

Without SQL, write the same filter in pseudocode **and** Python: using the parallel arrays of the Students table, output the names of all students in "9A" with a mark of 80 or more.

5. Procedures and functions

5.1 Procedures — the recipe card

Explanation. A procedure is a recipe card on the wall: `CALL` means *let's do the recipe now*. It performs a job and hands **nothing** back. `CALL PrintLine('*', 5)` runs the recipe with `Symbol = '*'` and `Times = 5`, printing `*****` — the ingredients (parameters) are handed in; the recipe works with whatever it is given (up to **three** parameters in the exam). The same card can be used again with different ingredients: `CALL PrintLine('#', 3)` prints `###`.

Examples. Draw a line of `Times` copies of `Symbol`:

Pseudocode:

```
PROCEDURE PrintLine(Symbol : CHAR, Times : INTEGER)
    DECLARE I : INTEGER
    DECLARE LineText : STRING
    LineText ← ""
    FOR I ← 1 TO Times
        LineText ← LineText + Symbol
    NEXT I
    OUTPUT LineText
ENDPROCEDURE
```

CALL PrintLine('*', 5)

```
**Python (with shortcut):**
```

def print_line(symbol, times):
line_text = ""
for i in range(1, times + 1):
line_text += symbol # += shortcut
print(line_text)

print_line('*', 5) # prints *****

```
**Flowchart (the subroutine box points to a separate chart):**
```

(Start) [LineText <- ""] [I <- 1]
| | |
+------------+------------------+
v
< I <= Times? > -- No --> / OUTPUT LineText / --> (Stop)
|
Yes
v
[LineText <- LineText + Symbol]
|
v
[I <- I + 1] (back up to the test)

The call site is one box:
+-------------------------+
| CALL PrintLine('*', 5) | (subroutine: double side bars)
+-------------------------+

> **Common errors**
> **CALL is for procedures only.** A function call is an **expression** (it hands a value back) — writing `CALL MaxOf(3, 4)` is wrong; use `OUTPUT MaxOf(3, 4)` or put it in an expression. In Python the "call" is just the name: `print_line('*', 5)`.

Practice questions

> **Q27.** **[3 marks]**
> Write a procedure `PrintLine(Symbol : CHAR, Times : INTEGER)` that outputs one line of `Times` copies of `Symbol`, and call it to print 10 stars. Provide the Python version too.

5.2 Functions — the blender

Explanation. A function is a blender: ingredients in (**parameters**), press the button (**call**), and a smoothie comes back (**return value**). `Square(3)` hands back 9, `Square(4)` hands back 16, and because the call is an expression it can live inside arithmetic: `OUTPUT Square(3) + Square(4)` prints **25**. The definition states what comes back (pseudocode: `RETURNS INTEGER`); Python's type follows whatever `return` sends.

Examples. A function that compares two values and hands back the larger:

Pseudocode:

```
FUNCTION MaxOf(Value1 : INTEGER, Value2 : INTEGER) RETURNS INTEGER
    IF Value1 > Value2
        THEN
            RETURN Value1
        ELSE
            RETURN Value2
    ENDIF
ENDFUNCTION
```

OUTPUT "The winner is ", MaxOf(72, 88) // prints: The winner is 88

```
**Python (with shortcut):**
```

def max_of(value1, value2):
return value1 if value1 > value2 else value2 # or a plain nested if

print("The winner is", max_of(72, 88))

```
**Flowchart (the function as a subroutine that returns):**
```

+-------------------------+ +-------------------------+
| [W <- MaxOf(72, 88)] | --> | FUNCTION MaxOf: |
+-------------------------+ | < Value1 > Value2? > |
| Yes -> RETURN Value1 |
(Continue with W = 88) | No -> RETURN Value2 |
+-------------------------+

> **Common errors**
> **The empty blender.** A Python function without `return` hands back `None` — silently. The program runs and the garbage appears far from the real bug. In pseudocode, forgetting `RETURN` inside a `FUNCTION` (or putting `RETURN` in a procedure that should return nothing) is the same slip.

Practice questions

Q28. **[4 marks]**

Write a function `IsEven(Number : INTEGER) RETURNS BOOLEAN` that returns `TRUE` when `Number` is even. Then use it in a program that inputs 10 integers and outputs how many are even. Write both languages.

Q29. **[3 marks]**

State exactly what this program outputs, show your working, and give the Python version.

```
FUNCTION Square(X : INTEGER) RETURNS INTEGER
    RETURN X * X
ENDFUNCTION
```

DECLARE N : INTEGER
N ← 3
OUTPUT Square(N) + Square(N + 1)

5.3 Local and global — notepad vs whiteboard

Explanation. A variable declared **inside** a subroutine is written in a **personal notepad**: it exists only there and is thrown away afterwards. A variable declared at the top of the program is the **classroom whiteboard** — visible everywhere. Run this story with real values: the whiteboard holds `Total = 5`; the procedure declares its **own** `Total`, sets it to 10 (notepad only), and finishes; the final `OUTPUT Total` reads the whiteboard — **5**. Python behaves identically: assigning a name inside a function creates a local.

Examples. The shadowing story in code:

Pseudocode:

```
DECLARE Total : INTEGER
Total ← 5
PROCEDURE AddUp
    DECLARE Total : INTEGER
    Total ← 10
ENDPROCEDURE
CALL AddUp
OUTPUT Total                    // prints 5
```

Python:

```
total = 5
```

def add_up():
total = 10 # a NEW local (the notepad)

add_up()
print(total) # prints 5

```
**Flowchart:**
```

(Start) [Total <- 5]
|
v
+--------------------+
| CALL AddUp | ---> PROCEDURE AddUp:
+--------------------+ [local Total <- 10] (notepad)
| (whiteboard untouched)
v
/ OUTPUT Total / -> 5
|
v
(Stop)

Common errors

The shadowing trap. If a subroutine declares a name that already exists globally, it gets a **fresh local** — changes inside never travel back out. To change a global's value, work through a **return value** (or in pseudocode, don't re-declare the name inside).

Practice questions

Q30. **[2 marks]**

What does this program output? Explain why, then show the Python equivalent and its output.

```
DECLARE Total : INTEGER
Total ← 5
PROCEDURE AddUp
    DECLARE Total : INTEGER
    Total ← 10
ENDPROCEDURE
CALL AddUp
OUTPUT Total
```

6. Files

6.1 File operations — the filing cabinet

Explanation. Variables live on the desk and are swept away when the program ends; a **file** is the filing cabinet that survives. `FOR READ` takes a folder out to copy pages; `FOR WRITE` inserts a **brand-new empty folder** — anything the old one held is gone (that is why writing overwrites, not appends). Work through real values: "Scores.txt" holds 40, 60, 80 — three reads fill the till 0 → 40 → 100 → 180, and the average is 180 ÷ 3 = **60**. Always `CLOSEFILE` — shut the drawer.

Examples. Read 3 marks from "Scores.txt" and output the average:

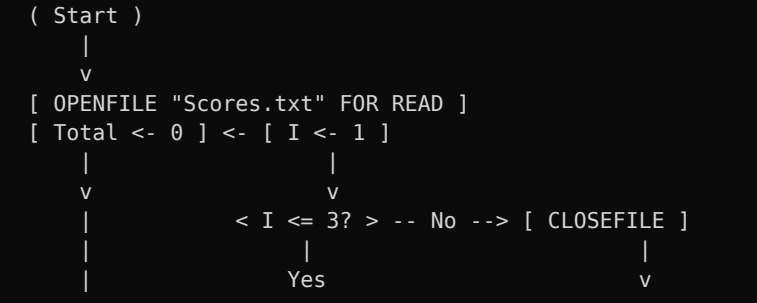
Pseudocode:

```
OPENFILE "Scores.txt" FOR READ
Total ← 0
FOR I ← 1 TO 3
    READFILE "Scores.txt", Mark
    Total ← Total + Mark
NEXT I
CLOSEFILE "Scores.txt"
OUTPUT "Average is ", Total / 3          // prints: Average is 60
```

Python (with shortcuts):

```
total = 0
f = open("Scores.txt", "r")              # "r" = FOR READ
for i in range(1, 4):
    mark = int(f.readline())
    total += mark
f.close()
print(f"Average is {total / 3}")         # f-string shortcut
```

Flowchart:



```
|                v                  / OUTPUT Total / 3 /
|      [ READFILE Mark ]                    |
|                |                          v
|                v                      ( Stop )
|      [ Total <- Total + Mark ]
|                |
+-- [ I <- I + 1 ] (back up to the test)
```

Common errors

Three file traps. (1) **Write erases:** `FOR WRITE` / `"w"` truncates the file first — it does not append; opening a file you meant to **keep** for writing destroys it. (2) **The open drawer:** forgetting `CLOSEFILE` / `f.close()` leaves later reads with stale or missing data. (3) **Mode mismatch:** never `WRITEFILE` to a file opened `FOR READ` (or write on `"r"`) — open the source for reading and the destination for writing, both at once.

Practice questions

Q31. **[5 marks]**

The file "Scores.txt" contains exactly 10 integer exam marks, one per line. Write an algorithm to output their average, in pseudocode **and** Python.

Q32. **[2 marks]**

State what happens to the existing contents of "Notes.txt" when this runs, and why. What is the Python equivalent?

```
OPENFILE "Notes.txt" FOR WRITE
```

Q33. **[3 marks]**

A student copies "Old.txt" into "New.txt" by opening "Old.txt" and writing to it with `WRITEFILE`. Explain what is wrong and give the corrected opening lines in both languages.

Q34. **[4 marks]**

Write a program that copies a single line of text from "FileA.txt" to "FileB.txt", in pseudocode **and** Python.

7. Standard algorithms and trace tables

7.1 Maximum, minimum and average — the tallest kid

Explanation. Finding the tallest kid in a line: assume the **first** kid is tallest, then walk the line updating the record whenever someone beats it. Real values: `A = [3, 9, 4]`: start `Max = 3`; look at 9 — bigger, `Max = 9`; look at 4 — smaller, ignored. Answer **9**. Minimum is the same walk with <. Average is the till divided by the count: (3+9+4) ÷ 3 = **5.33...**. The first value must seed `Max`/`Min` — starting at 0 breaks for negative numbers (a line of negative temperatures would wrongly report max = 0).

Examples. Largest and smallest of 3 values:

Pseudocode:

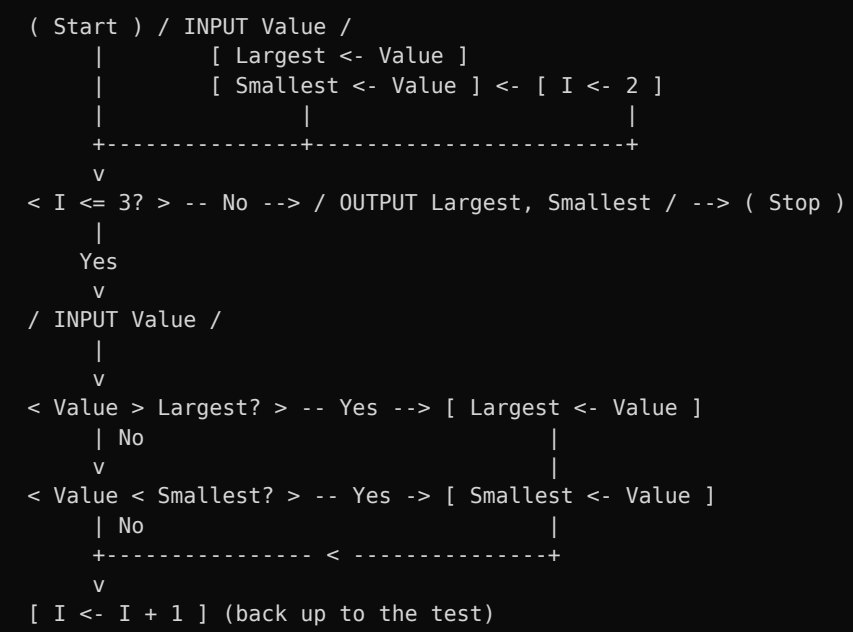
```
INPUT Value
Largest ← Value
Smallest ← Value
FOR I ← 2 TO 3
    INPUT Value
    IF Value > Largest
        THEN
            Largest ← Value
```

```
    ENDIF
    IF Value < Smallest
        THEN
            Smallest ← Value
    ENDIF
NEXT I
OUTPUT Largest, Smallest
```

Python (with shortcuts):

```
value = int(input("Number: "))
largest = smallest = value
for i in range(2, 4):
    value = int(input("Number: "))
    if value > largest:
        largest = value
    if value < smallest:
        smallest = value
print(largest, smallest)
# shortcut when the method itself is NOT being tested:
# print(max(a), min(a), sum(a) / len(a))
```

Flowchart:



Common errors

Seeding and direction. Seed `Max`/`Min` from the **first value**, not 0. And check the update direction: in a smallest-value search, `IF Value > Smallest` silently finds the biggest instead. Python shortcut caveat: `max(a)` is great code — but if the question says **"write an algorithm to find the maximum"**, the loop *is* the answer.

Practice questions

Q35. **[5 marks]**

Write an algorithm in pseudocode **and** Python that inputs 12 integers and outputs the **largest** and the **smallest** of them.

Q36. **[2 marks]**

A question says: "Write an algorithm to find the maximum value in an array." A student answers `print(max(a))` in Python. Discuss whether this earns the marks.

7.2 Linear search — the register scan

Explanation. Run your finger down a class register name by name until you find the target — or reach the bottom. Real values: `A = [7, 2, 9, 4]`, searching for 9: look at locker 1 (7 ≠ 9), locker 2 (2 ≠ 9), locker 3 (9 = 9 — **found at position 3**), stop; locker 4 is never visited. In Python the same search reports index **2** (0-based). If the target is absent, the finger reaches the end and the answer is "not found" — the Found flag remembers which outcome happened.

Examples. Search for `SearchKey` and report the position:

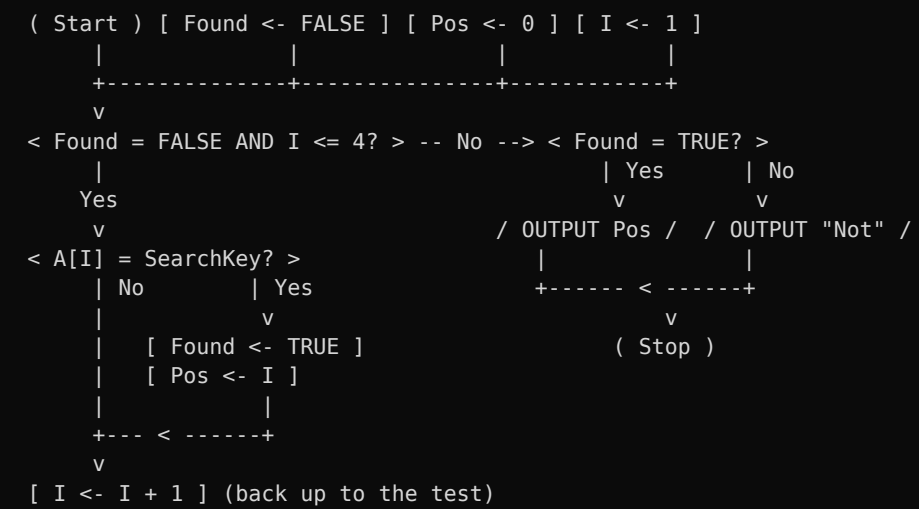
Pseudocode:

```
Found ← FALSE
Position ← 0
I ← 1
WHILE Found = FALSE AND I <= 4 DO
    IF A[I] = SearchKey
        THEN
            Found ← TRUE
            Position ← I
    ENDIF
    I ← I + 1
ENDWHILE
IF Found = TRUE
    THEN
        OUTPUT "Found at ", Position
    ELSE
        OUTPUT "Not found"
ENDIF
```

Python (with shortcuts; positions 0-based):

```
found = False
position = 0
i = 0
while found == False and i < 4:
    if a[i] == search_key:
        found = True
        position = i
    i += 1
print(f"Found at {position}" if found else "Not found")
```

Flowchart:



Common errors

Two positions, two bases. The same search reports position 3 in pseudocode and index 2 in Python — answer in the language of the question. And the loop must **stop when found** (the `Found = FALSE` in the condition); scanning on after a find can overwrite the position with a later match.

Practice questions

Q37. **[4 marks]**

A holds 7, 2, 9, 4 in positions 1 to 4 and SearchKey is 9. Complete a trace table (columns `I`, `A[I]`, `Found`, `Position`) for the pseudocode above, and state the output.

Q38. **[3 marks]**

Write a linear search in pseudocode **and** Python that inputs a name and a list of 5 names, then outputs "Yes" or "No" depending on whether the name is in the list.

7.3 Bubble sort — the fizzy drink

Explanation. A line of kids sorts by height by repeatedly walking the line, comparing **neighbours** and swapping any pair out of order. Each full walk floats the tallest remaining "bubble" to the end. Real values, sorting [5, 3, 4]: walk 1 compares 5 vs 3 (swap → [3, 5, 4]), then 5 vs 4 (swap → [3, 4, 5]); walk 2 compares 3 vs 4 (no swap), 4 vs 5 (no swap). A walk with **no swaps** means everyone is in order — stop. The inner loop compares `A[Inner]` with `A[Inner + 1]`, so it must stop **one locker early**.

Examples. Sort `A[1]` to `A[N]` ascending:

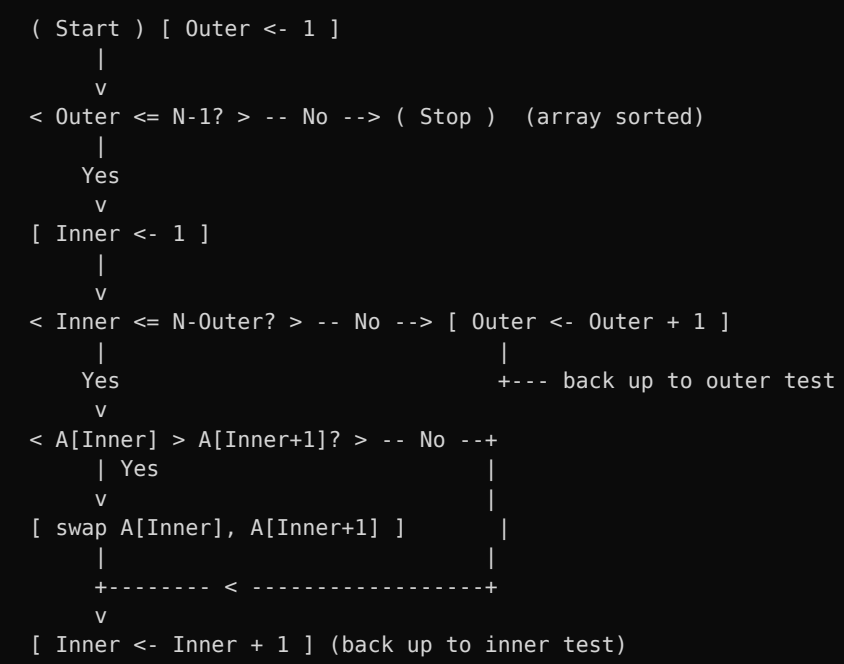
Pseudocode:

```
FOR Outer ← 1 TO N - 1
    FOR Inner ← 1 TO N - Outer
        IF A[Inner] > A[Inner + 1]
            THEN
                Temp ← A[Inner]
                A[Inner] ← A[Inner + 1]
                A[Inner + 1] ← Temp
        ENDIF
    NEXT Inner
NEXT Outer
```

Python (with shortcuts):

```
for outer in range(0, n - 1):
    for inner in range(0, n - 1 - outer):
        if a[inner] > a[inner + 1]:
            a[inner], a[inner + 1] = a[inner + 1], a[inner]   # swap shortcut
```

Flowchart (one pass, simplified):



Common errors

The neighbour beyond the end. The inner loop going `TO N` (Python `range(0, n)`) reads `A[N + 1]` on its last run — a locker that doesn't exist (Python: IndexError). The bound is `N - Outer` (or at least `N - 1`). Also note the swap needs the temporary `Temp` in pseudocode; Python's `a[i], a[j] = a[j], a[i]` does it in one line.

Practice questions

Q39. [5 marks]

Trace one complete bubble sort of [5, 3, 4] (ascending). Show the array after every comparison and swap, and say how you know the sort is finished.

Q40. [3 marks]

This bubble sort should sort `A[1]` to `A[N]` but sometimes crashes. Identify the error, write the corrected inner loop line, and show the same bug in Python form.

```
FOR Outer ← 1 TO N - 1
    FOR Inner ← 1 TO N
        IF A[Inner] > A[Inner + 1]
```

7.4 Trace tables — the flight recorder

Explanation. A trace table is the flight recorder: one **column per variable** (plus OUTPUT), one **row per step**, the initial values first. It works identically for pseudocode and Python, because the logic — not the syntax — is what moves. Worked with real values, tracing `Total ← 0; Count ← 0; REPEAT INPUT Value ... UNTIL Value = 0` with inputs 5, −3, 2, 0:

Step	Value	Total	Count
before loop	-	0	0
1	5	5	1
2	-3	5	1
3	2	7	2
4	0	7	2

Read the story: 5 goes on the till and clicks the counter (5, 1); −3 fails the `> 0` test (unchanged); 2 adds (7, 2); 0 is processed, then the `UNTIL Value = 0` test stops the loop. Output: **7, 2**.

Common errors

Recorder mistakes. Put the initial row first; show each row **after** that step's changes; keep types consistent (no "TRUE" in a number column); and remember `UNTIL` is tested **after** the body — the row for the stopping value (0 here) is still written.

Practice questions

Q41. [4 marks]

Complete a trace table (`Value`, `Total`, `Count`) for the REPEAT program above with inputs 5, −3, 2, 0, and state what is output.

Q42. [2 marks]

Explain why programmers use dry-runs and trace tables before running a program.

8. Program development and testing

8.1 The PDLC and validation checks — the bouncer

Explanation. Building software has four stages: **analysis** (what is needed? decompose the problem), **design** (structure diagrams, flowcharts, pseudocode), **coding** (write it, test as you go), **testing** (run it against test data). Validation is the **bouncer** at the door of your program: input data must pass the checks before it gets inside. The six checks with real values — a registration form wanting a 6-letter username and an age of 13–19: **presence** (is anything typed?), **length** (exactly 6 characters?), **format** (letters only?), **type** (age is a number), **range** (13 ≤ age ≤ 19 — 15 passes, 20 is bounced), **check digit** (like the last digit of a credit card: recalculated and compared).

Examples. The bouncer as code:

Pseudocode:

```
REPEAT
    INPUT Age
    IF Age < 13 OR Age > 19
        THEN
            OUTPUT "Invalid age - must be 13 to 19"
    ENDIF
UNTIL Age >= 13 AND Age <= 19
OUTPUT "Welcome"
```

Python (with shortcuts):

```
while True:
    age = int(input("Age: "))
    if age < 13 or age > 19:
        print("Invalid age - must be 13 to 19")
    else:
        break                       # passed the bouncer
print("Welcome")
```

Flowchart:

```
 ( Start )
     |
     v
 / INPUT Age /
     |
     v
< 13 <= Age <= 19? > -- Yes --> / OUTPUT "Welcome" / --> ( Stop )
     |
     No
     v
 / OUTPUT "Invalid" /
     |
     +------ back up to INPUT
```

Common errors

Validation vs verification. Validation (bouncer) asks "is this data **reasonable**?"; verification (proofreader) asks "was it **copied accurately**?" — comparing two entries of the same password is **verification**, not validation. And validate **before** processing; a program that averages the marks first and checks them later has already crashed.

Practice questions

Q43. **[3 marks]**

A registration program requires a username that is **exactly 6 characters long** and **letters only**. Identify the validation checks needed and say what each one checks.

Q44. **[3 marks]**

Define **validation** and **verification**, making the difference between them clear.

8.2 Verification and test data — the proofreader's checklist

Explanation. Verification catches **entry** errors: a **visual check** (re-read the typed data against the source) or **double entry** (type it twice; if "Secret123" then "Secretl23", the comparison flags a mismatch). Test data proves the program survives every kind of impact: for age 13–19 — **normal** 15 (accepted), **abnormal** "abc" (rejected: wrong type), **extreme** 13 and 19 (the smallest and largest **accepted**), **boundary** 12, 13, 19, 20 (the extremes **plus** the adjacent **rejected** values). Boundary data is where off-by-one bugs die: a program accepting 12 has a <= where it needs <.

Examples. Double entry as code:

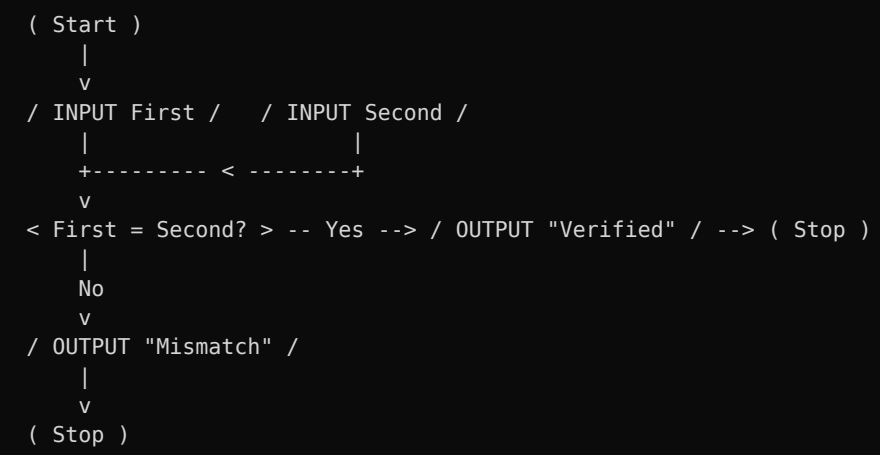
Pseudocode:

```
INPUT First
INPUT Second
IF First = Second
    THEN
        OUTPUT "Entry verified"
    ELSE
        OUTPUT "Mismatch - retype"
ENDIF
```

Python:

```
first = input("Type it: ")
second = input("Type it again: ")
if first == second:
    print("Entry verified")
else:
    print("Mismatch - retype")
```

Flowchart:



Common errors

Extreme vs boundary. Extreme = the largest/smallest value that should be **accepted** (13, 19). **Boundary** = those values **plus** the neighbours that must be **rejected** (12, 20). Answering "13 and 19" for boundary data loses half the marks — the rejected neighbours are the point.

Practice questions

Q45. **[4 marks]**

A competition form accepts only ages between 13 and 19 inclusive. Suggest suitable test data of each type: (a) normal (b) abnormal (c) extreme (d) boundary.

Q46. **[3 marks]**

Classify each task by PDLC stage (analysis, design, coding or testing): (a) identifying what data the program must store (b) drawing a structure diagram (c) running the program with sample data to find errors.

9. Flowcharts

9.1 Symbols and conversion — the road map

Explanation. A flowchart is the algorithm as a **road map**: boxes are places, arrows are roads, and a diamond is a fork with a yes/no sign. There are exactly **six** legal shapes: **Terminator** (rounded: Start/Stop), **Process** (rectangle: something done), **Decision** (diamond: one question, **two** labelled exits), **Input/Output** (parallelogram), **Subroutine** (rectangle with double side bars: a call to a separate chart), and **Flow line** (arrow). Loops are roads that circle back — no special "loop box". Real values: the password loop below asks the question, and the No road curves back to the input.

Examples. The password loop in all three forms:

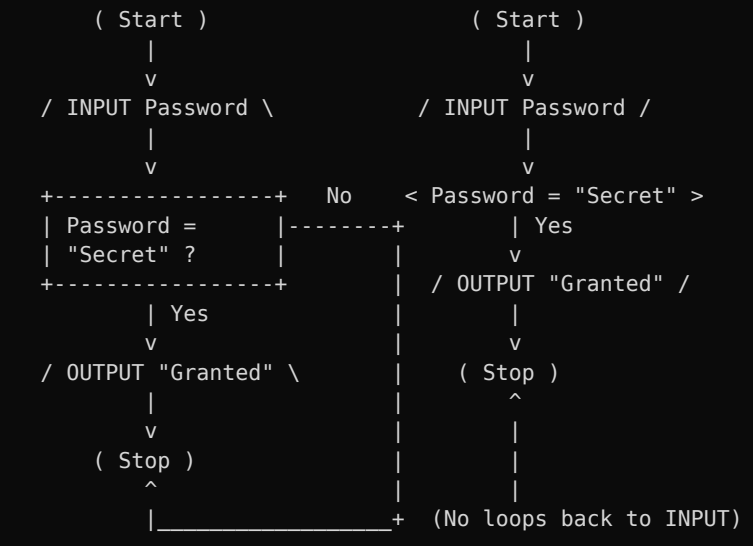
Pseudocode:

```
REPEAT
    INPUT Password
UNTIL Password = "Secret"
OUTPUT "Access granted"
```

Python (with shortcut):

```
while True:
    password = input("Password: ")
    if password == "Secret":
        break
print("Access granted")
```

Flowchart:



(Diamond or box? The middle shape must be a **diamond** — draw decisions as diamonds, actions as rectangles.)

Common errors

Shape crimes. A decision drawn as a parallelogram or rectangle is wrong (diamond!). Unlabelled decision exits are wrong (Yes/No or True/False on **both**). Assignments hiding in input/output parallelograms are wrong (`X ← X + 1` is a Process). Missing flow-line arrowheads leave the reader guessing the direction.

Practice questions

Q47. **[4 marks]**

Draw a flowchart for: *Start. Repeatedly ask for a password until the user enters "Secret". Then output "Access granted" and stop.* Describe each box, its shape, and where every flow line goes.

Q48. **[4 marks]**

Describe the flowchart for this pseudocode, naming each box's shape and contents, and saying where the loop line goes.

```
Total ← 0
FOR I ← 1 TO 3
    INPUT Value
    Total ← Total + Value
NEXT I
OUTPUT Total
```

Q49. **[2 marks]**

Sam drew a decision as a parallelogram, and left one exit of the decision unlabelled. State what is wrong in each case and how to fix it.

10. Boolean logic and logic gates

10.1 The seven gates — tiny decision machines

Explanation. A gate is a machine with one or two input levers and a bulb. With real values `A = 1, B = 0`: **AND** (two-key safe) = 0 (both keys must turn); **OR** (two doors) = 1 (one door is enough); **XOR** (one slice each, not both) = 1 (they differ); **NAND** = 1 (AND inverted); **NOR** = 0 (OR inverted); **NOT A** = 0 (opposite day). NOT takes **one** input; all others take **two**. A truth table lists **every combination** of the levers — 2 inputs = 4 rows, 3 inputs = 8 rows, like the outcomes of flipping coins.

Examples. `X = (A AND B) OR C` — *the fan runs when it is hot AND the window is open, OR when the manual switch is on*:

A	B	C	A AND B	X = (A AND B) OR C
0	0	0	0	0
0	0	1	0	1
0	1	0	0	0
0	1	1	0	1
1	0	0	0	0
1	0	1	0	1
1	1	0	1	1
1	1	1	1	1

Pseudocode:

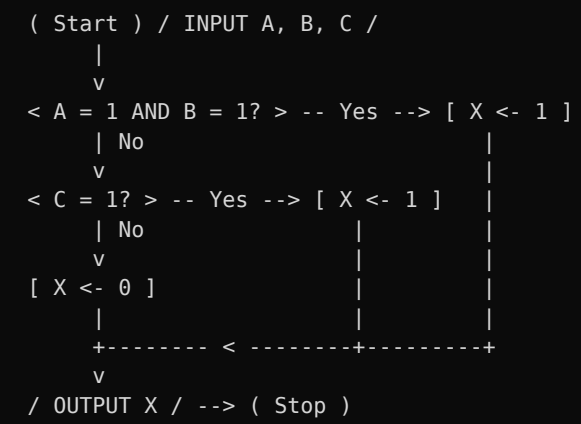
```
INPUT A
INPUT B
INPUT C
X ← (A AND B) OR C
OUTPUT X
```

Python (with shortcut):

```
a = int(input("A: "))     # 0 is False, 1 is True
b = int(input("B: "))
```

```
c = int(input("C: "))
x = (a and b) or c
print(int(x))
```

Flowchart (the expression as roads):



Common errors

Parentheses and rows. Write expressions with explicit parentheses — `(A AND NOT B) OR C` — pseudocode has no precedence rules for mixed logical operators. And count the rows: a 3-input truth table **always** has 8 rows (000 to 111); a 6-row table is missing combinations.

Practice questions

Q50. **[3 marks]**

Complete the truth table for `X = (A OR B) AND NOT C`.

Q51. **[2 marks]**

Compare the behaviour of the OR and XOR gates.

10.2 Conversions — statement, expression, circuit, truth table

Explanation. The exam moves freely between four forms of the same logic: **statement** ("X is TRUE only when exactly one of A, B is TRUE"), **expression** (`X = (A AND NOT B) OR (B AND NOT A)`), **circuit** (an AND on A and NOT-B, another on B and NOT-A, both feeding an OR), and **truth table** (every combination). The key move is translating word by word: "only when" tells you the output rows; "and" = AND gate; "or" = OR gate; "not" = NOT gate. Circuits are drawn **exactly as stated — never simplified** (max 3 inputs, 1 output).

Common errors

Simplifying. If the statement says "(A AND B) OR (A AND C)", draw **both** AND gates and the OR — combining them into "A AND (B OR C)" loses marks, even though it is logically equal. And when building a truth table from a circuit, evaluate **one gate at a time** into an extra column (like the `A AND B` column above) before combining.

Practice questions

Q52. **[3 marks]**

Write a logic expression (and name the gate) for: *X is TRUE only when exactly one of the inputs A and B is TRUE.*

Q53. **[3 marks]**

Describe the logic circuit for `X = A AND NOT B` (gates and connections).

Q54. **[4 marks]**

Complete the truth table for `X = (A AND B) OR (NOT C)`.

11. The scenario question

Explanation. The last question of Paper 2 is a **15-mark unseen scenario** (~30 minutes). You may answer in **pseudocode, Python, Visual Basic or Java** — everywhere else on the paper, pseudocode only. The marks follow the **logic** (every requirement covered, correct structure, validation where implied) and **maintainability** (meaningful names, comments) — not language tricks. Plan first: name your variables, sketch the main loop (a flowchart sketch on the question paper is allowed and wise), decide on one or two functions, then write. Sanity-check each part with mental test data — a normal value and a boundary value.

Examples. *Ask for 5 daily temperatures; output the average and the number of days above 25°.* With real values 20, 26, 30, 24, 25: total 125, average **25**, days above 25: **2** (26 and 30).

Pseudocode:

```
DECLARE Temps : ARRAY[1:5] OF REAL
FOR Day ← 1 TO 5
    INPUT Temps[Day]
NEXT Day
Total ← 0
FOR I ← 1 TO 5
    Total ← Total + Temps[I]
NEXT I
Average ← Total / 5
OUTPUT "Average temperature: ", Average
HotDays ← 0
FOR I ← 1 TO 5
    IF Temps[I] > 25
        THEN
            HotDays ← HotDays + 1
    ENDIF
NEXT I
OUTPUT "Days above 25 degrees: ", HotDays
```

Python (with shortcuts):

```
temps = []
for day in range(1, 6):
    temps.append(float(input(f"Temperature for day {day}: ")))
```

total = 0
for value in temps:
total += value
average = total / 5
print(f"Average temperature: {average}")

hot_days = sum(1 for v in temps if v > 25) # or a plain counting loop
print(f"Days above 25 degrees: {hot_days}")

```
**Flowchart (the program's spine):**
```

(Start) [I <- 1]
|
v
< I <= 5? > -- No --> [Total <- 0, I <- 1] ...

| Yes |
v v
/ INPUT Temps[I] / < I <= 5? > -- No --> / OUTPUT Average /
| | Yes |
v v v
[I <- I + 1] [Total <- Total + Temps[I]] < count HotDays loop >
| |
v v
[I <- I + 1] / OUTPUT HotDays /

Practice questions

Q55. **[10 marks]**

Scenario practice. The school canteen records meals sold each day for 10 days. Write a complete program that: asks the user for each day's sales, **rejecting and re-asking on any negative value**; stores the 10 values; outputs the **total** and the **average**; outputs the **highest**; and outputs **how many days were above the average**. Write it in either pseudocode or Python (in the exam, either is allowed for this question only).

Answers

Every answer gives the **result**, the **working** (why it is what it is), and the **mark points**. Check your method against the working — the method is what earns marks in the exam.

Section 2 — Iteration and nested constructs

Answer 1 (3 marks)
Pseudocode:

```
Total ← 0
FOR I ← 1 TO 10
    Total ← Total + I ^ 2
NEXT I
OUTPUT Total
```

Python:

```
total = 0
for i in range(1, 11):
    total = total + i ** 2
print(total)
```

Why: the till starts at 0; each lap adds the square of the lap number (1 + 4 + 9 + ... + 100 = 385). The loop counter is the number being squared — no literal 385 anywhere.
Mark points: initialising `Total` (1); loop 1 to 10 accumulating the square of the counter (1); output of the calculated total (1). Accept `I * I` / `i * i`.

Answer 2 (2 marks)
Outputs `3, 5, 7` (three lines). STEP 2 visits 3, 5, 7 — then 9 would exceed the end value 7, so the loop stops (the fence-post rule: `TO 7` includes 7).
Python: `for i in range(3, 8, 2): print(i)` — the end is 8 because range stops **before** its end value.
Mark points: correct output 3, 5, 7 (1); correct `range(3, 8, 2)` (1).

Answer 3 (3 marks)
Pseudocode:

```
I ← 1
WHILE I <= 10 DO
    OUTPUT I * 2
    I ← I + 1
```

```
ENDWHILE
```

Python:

```
i = 1
while i <= 10:
    print(i * 2)
    i = i + 1
```

Why: a WHILE loop needs everything the FOR gave for free: the counter initialised before the loop (`I ← 1`), the test (`I <= 10` — inclusive, or you lose the lap where I = 10), and the manual increment (the hamster-wheel rule).
Mark points: initialise the counter (1); correct inclusive condition (1); increment and correct WHILE structure (1).

Answer 4 (2 marks)
The condition `I < 5` should be `I <= 5` — with < the loop stops when `I` reaches 5, so 5 is never printed (one fence post too few). Python: `while i <= 5:`.
Mark points: identifying the wrong condition (1); correcting it in both languages (1).

Answer 5 (3 marks)
`UNTIL` stops when the condition is **TRUE**, so this loop stops when the mark is **invalid** (`Mark < 0 OR Mark > 100` becomes true) and keeps asking while it is valid — exactly inverted. The stopping state must be "valid":

```
REPEAT
    INPUT Mark
UNTIL Mark >= 0 AND Mark <= 100
```

Mark points: explaining the UNTIL polarity (1); corrected condition with both bounds and `AND` (1); correct final code (1).

Answer 6 (2 marks)
(a) **FOR** — 20 marks means 20 known laps; the count is fixed before the loop starts.
(b) **REPEAT ... UNTIL** — at least one question must be asked (the body runs once minimum), and the loop stops when the password becomes "Open123".
(c) **WHILE** — the box is checked **before** each slice; if it starts empty, zero slices are eaten.
Mark points: one per situation with a fitting justification (award 2 of 3 for one correct and one partially justified).

Answer 7 (3 marks)
Error 1: `Total` is never initialised — the till starts at garbage; it needs `Total ← 0` before the loop.
Error 2: `TO 4` stops at 4 (fence post) while the question wants 1 to 5.

```
Total ← 0
FOR I ← 1 TO 5
    Total ← Total + I
NEXT I
OUTPUT Total
```

The corrected code prints 15.
Mark points: missing initialisation (1); wrong upper bound (1); corrected code (1).

Answer 8 (3 marks)
Pseudocode:

```
Count ← 0
FOR I ← 1 TO 5
    INPUT Value
    IF Value > 10
        THEN
            Count ← Count + 1
    ENDIF
NEXT I
OUTPUT Count
```

Python:

```
count = 0
for i in range(1, 6):
    value = int(input("Number: "))
    if value > 10:
        count += 1
print(count)
```

Why: a counter (clicker) with the IF as the test — one click per value over 10. The counter starts at 0.
Mark points: counter initialised (1); loop of 5 inputs (1); correct test, increment and output (1).

Answer 9 (3 marks)
Pseudocode:

```
IF Age < 13
    THEN
        OUTPUT "Child"
    ELSE
        IF Age < 20
            THEN
                OUTPUT "Teen"
            ELSE
                OUTPUT "Adult"
        ENDIF
ENDIF
```

Python: the same nested inside else (or the elif ladder). Age 10 → **Child** (fails the first test's bound... 10 < 13 → Child); Age 15 → **Teen** (not < 13, but < 20); Age 22 → **Adult** (neither).
Mark points: correct nested structure (1); correct outputs for 10, 15, 22 (1); correct ELSE logic — the second test only applies to 13-and-overs (1).

Answer 10 (3 marks)

```
count = 0
for mark in marks:
    if mark >= 80:
        count += 1
        print("Distinction")
    elif mark >= 50:
        count += 1
        print("Pass")
    else:
        print("Fail")
print(count)
```

Why: the three shortcuts are for-each (`for mark in marks` — no index needed), += (`count += 1` twice), and elif (the nested IF flattened; identical logic, tested top to bottom).
Mark points: one per shortcut applied correctly.

Answer 11 (2 marks)
Prints `15 0`. The loop adds 5 + 4 + 3 + 2 + 1 = 15 while counting n down to 0; `total += n` is `total = total + n`, `n -= 1` is `n = n - 1`; the loop stops because `n > 0` finally fails.
Pseudocode equivalent:

```
N ← 5
Total ← 0
WHILE N > 0 DO
    Total ← Total + N
    N ← N - 1
ENDWHILE
OUTPUT Total, N
```

Mark points: 15 (1); 0 (1).

Section 3 — Strings

Answer 12 (3 marks)
(a) **8** (b) **"mpu"** (c) **"O"**.
Python: `len(text)` → 8; `text[2:5]` → "mpu"; `text[1:2].upper()` → "O".
Why: positions are C=1, o=2, m=3, p=4, u=5 … `SUBSTRING(Text, 3, 3)` starts at bead 3 ("m") and takes 3 beads: "mpu". `SUBSTRING(Text, 2, 1)` is "o", upper-cased to "O". Python beads are numbered one lower: position 3 = index 2.
Mark points: one per part.

Answer 13 (2 marks)
`SUBSTRING("HELLO", 2, 3)` = **"ELL"** (beads 2, 3, 4). LENGTH of that = **3**.
Python: `"HELLO"[1:4]` — start at index 1 (bead 2), stop before index 4.
Mark points: "ELL" (1); length 3 and correct slice (1).

Answer 14 (4 marks)
Pseudocode:

```
INPUT S
Count ← 0
FOR I ← 1 TO LENGTH(S)
    Ch ← LCASE(SUBSTRING(S, I, 1))
    IF Ch = "e"
        THEN
            Count ← Count + 1
    ENDIF
NEXT I
OUTPUT Count
```

Python:

```
s = input("Text: ")
count = 0
for ch in s:
    if ch.lower() == "e":
        count += 1
print(count)
```

Why: one walk of the necklace, one look per bead; normalising the case (LCASE/lower) makes "E" and "e" count together. (Python shortcut `if ch.lower() in "e":` also works but == is clearer here.)
Mark points: loop over the whole string (1); extracting each character (1); case-insensitive comparison (1); counting and outputting (1).

Answer 15 (3 marks)
Pseudocode:

```
INPUT S
Reversed ← ""
FOR I ← LENGTH(S) TO 1 STEP -1
    Reversed ← Reversed + SUBSTRING(S, I, 1)
NEXT I
OUTPUT Reversed
```

Python:

```
s = input("Text: ")
reversed_text = ""
for i in range(len(s) - 1, -1, -1):
    reversed_text += s[i]
print(reversed_text)
```

Why: walk the necklace from the **last** bead to the first (`STEP -1`), stitching each bead onto the growing string. "ABC" → "C" → "CB" → "CBA". `print(s[::-1])` is the accepted Python shortcut.
Mark points: downward loop (1); character extraction (1); concatenation and output (1).

Answer 16 (3 marks)
(a) `print("-" * 30)` (b) `if ch.upper() in "AEIOU":` (c) `print(s[::-1])`.

Why: repetition (*) builds the dashes; `in` replaces the long OR chain ("is this bead in the vowel set?"); `[::-1]` is the reverse slice.
Mark points: one per task.

Answer 17 (2 marks)
Output: `8 mpu COMPUTER` (one line). `len("Computer")` = 8; `s[2:5]` = indexes 2, 3, 4 = "mpu"; `s.upper()` = "COMPUTER".
Pseudocode version:

```
S ← "Computer"
OUTPUT LENGTH(S), SUBSTRING(S, 3, 3), UCASE(S)
```

Mark points: 8 and "mpu" (1); "COMPUTER" and correct pseudocode (1).

Section 4 — Arrays and data tables

Answer 18 (3 marks)
Pseudocode:

```
DECLARE Marks : ARRAY[1:20] OF INTEGER
FOR I ← 1 TO 20
    INPUT Marks[I]
NEXT I
```

Python:

```
marks = []
for i in range(0, 20):
    marks.append(int(input("Mark: ")))
```

Mark points: declaration of 20 integers (1); loop 20 times (1); input into the element using the counter as index (1).

Answer 19 (4 marks)
T:
Step	I	A[I]	Max
before loop	-	-	2
1	2	7	7
2	3	4	7
3	4	9	9
Output: 9.
Why: Max is seeded from locker 1 (value 2); 7 beats it (Max = 7); 4 loses (unchanged); 9 beats it (Max = 9). The Python twin is identical with `a = [2, 7, 4, 9]`.
Mark points: Max starts at 2 (1); one mark per correct trace row (3).

Answer 20 (4 marks)
Error 1 — index out of range: `range(1, 4)` visits 1, 2, 3 but a 3-item list has indexes 0, 1, 2, so `scores[3]` asks for a locker that doesn't exist (IndexError). Fix: `range(0, 3)` or `range(len(scores))`.
Error 2 — text plus number: `"Total: " + total` joins text to an int (TypeError). Fix: `print("Total:", total)`.

```
scores = [50, 70, 90]
total = 0
for i in range(0, 3):
    total = total + scores[i]
print("Total:", total)
```

Prints `Total: 210`.
Mark points: each error identified (2); each fix shown (2).

Answer 21 (4 marks)
Pseudocode:

```
FOR Row ← 1 TO 2
    Sum ← 0
```

```
    FOR Col ← 1 TO 3
        Sum ← Sum + Grid[Row, Col]
    NEXT Col
    OUTPUT "Row ", Row, " total is ", Sum
NEXT Row
```

Python:

```
for row in range(0, 2):
    total = 0
    for col in range(0, 3):
        total += grid[row][col]
    print("Row", row + 1, "total is", total)
```

Outputs `Row 1 total is 8` (3+1+4) then `Row 2 total is 13` (2+5+6).
Why: the till (`Sum`) is reset **inside the outer loop** — one receipt per row — and the inner loop scans every seat in that row.
Mark points: nested loop covering both dimensions (2); accumulation with both indices (1); correct totals output inside the outer loop (1).

Answer 22 (2 marks)
Pseudocode:

```
DECLARE Squares : ARRAY[1:5] OF INTEGER
FOR I ← 1 TO 5
    Squares[I] ← I ^ 2
NEXT I
```

Python:

```
squares = []
for i in range(1, 6):
    squares.append(i * i)
```

Accept `squares = [i * i for i in range(1, 6)]`.
Mark points: correct storage (1); loop 1 to 5 squaring the counter (1).

Answer 23 (3 marks)

```
SELECT Name FROM Students WHERE Class = "9A" AND Mark >= 80 ORDER BY Mark DESCENDING
```

Why: both conditions belong in one `WHERE` joined with `AND`; `ORDER BY ... DESCENDING` runs the results highest-first.
Mark points: SELECT Name FROM Students (1); WHERE with AND (1); ORDER BY Mark DESCENDING (1).

Answer 24 (3 marks)
Output: **3**.
Why: every row is checked against **both** conditions: Aisha (9A, 84 ✓), Ben (9B ✗ — class fails despite 72), Chen (9A, 91 ✓), Dina (65 ✗), Elias (9A, 78 ✓). Three records survive; COUNT counts them.
Mark points: correct output 3 (2); correctly identifying the matching records (1).

Answer 25 (2 marks)
`StudentID` — unique to every student, always present, and never changes (a name can repeat or be corrected; a date of birth is shared by twins).
Mark points: correct field (1); justification (1).

Answer 26 (3 marks)
Pseudocode:

```
FOR I ← 1 TO 5
    IF Class[I] = "9A" AND Mark[I] >= 80
        THEN
            OUTPUT Name[I]
    ENDIF
NEXT I
```

Python:

```
for i in range(0, 5):
```

```
    if class_[i] == "9A" and mark[i] >= 80:
        print(name[i])
```

Why: a query is a filter loop over the parallel arrays — exactly the WHERE logic written out by hand. (Ordering would need the bubble sort of section 7.3 first.)
Mark points: loop over the rows (1); both conditions (1); correct output of the name (1).

Section 5 — Procedures and functions

Answer 27 (3 marks)
Pseudocode:

```
PROCEDURE PrintLine(Symbol : CHAR, Times : INTEGER)
    DECLARE I : INTEGER
    DECLARE LineText : STRING
    LineText ← ""
    FOR I ← 1 TO Times
        LineText ← LineText + Symbol
    NEXT I
    OUTPUT LineText
ENDPROCEDURE
```

CALL PrintLine('*', 10)

```
**Python:**
```

def print_line(symbol, times):
line_text = ""
for i in range(1, times + 1):
line_text += symbol
print(line_text)

print_line('*', 10)

```
Prints ********** (10 stars on one line).
Mark points: header with both typed parameters (1); loop building the string (1); single output and correct cal
```

Answer 28 (4 marks)
Pseudocode:

```
FUNCTION IsEven(Number : INTEGER) RETURNS BOOLEAN
    RETURN MOD(Number, 2) = 0
ENDFUNCTION
```

Count ← 0
FOR I ← 1 TO 10
INPUT Value
IF IsEven(Value)
THEN
Count ← Count + 1
ENDIF
NEXT I
OUTPUT Count

```
**Python:**
```

def is_even(number):
return number % 2 == 0

count = 0
for i in range(1, 11):
value = int(input("Number: "))
if is_even(value):
count += 1
print(count)

```
**Why:** a number is even exactly when the remainder on division by 2 is zero (__MOD(Number, 2) = 0__ / `number
```

```
Mark points: function header with return type (1); remainder test (1); loop of 10 inputs (1); counting and outp
```

Answer 29 (3 marks)
Outputs 25. Working: `Square(3)` = 3 × 3 = **9**; `Square(4)` = 4 × 4 = **16**; 9 + 16 = **25**. Function calls are expressions — each call is replaced by its returned value before the addition.
Python:

```
def square(x):
    return x * x

n = 3
print(square(n) + square(n + 1))
```

```
Mark points: 9 (1); 16 (1); 25 (1).
```

Answer 30 (2 marks)
Outputs 5. The `Total` declared inside the procedure is a **local** (the notepad) shadowing the global (the whiteboard); setting it to 10 changes only the notepad, which is discarded when the procedure ends.
Python equivalent:

```
total = 5

def add_up():
total = 10 # new local

add_up()
print(total) # prints 5
```

```
Mark points: correct output (1); explanation via local vs global (1).
```

Section 6 — Files

Answer 31 (5 marks)
Pseudocode:

```
OPENFILE "Scores.txt" FOR READ
Total ← 0
FOR I ← 1 TO 10
    READFILE "Scores.txt", Mark
    Total ← Total + Mark
NEXT I
CLOSEFILE "Scores.txt"
OUTPUT "Average is ", Total / 10
```

Python:

```
total = 0
f = open("Scores.txt", "r")
for i in range(10):
    mark = int(f.readline())
    total += mark
f.close()
print(f"Average is {total / 10}")
```

Why: the till accumulates the 10 reads (0 → … → sum of all marks), then divides by 10. The average must be computed **before** the file is closed but **after** the loop.
Mark points: open for reading (1); loop 10 times (1); read and accumulate (1); divide by 10 and output (1); close the file (1).

Answer 32 (2 marks)
The existing contents are **erased** — `FOR WRITE` inserts a brand-new empty folder; it does not append. Anything previously in "Notes.txt" is lost.
Python equivalent: `f = open("Notes.txt", "w")` — which likewise truncates the file.
Mark points: data lost/overwritten (1); because write mode creates a new file rather than adding (1).

Answer 33 (3 marks)
A file opened `FOR READ` cannot be written to with `WRITEFILE` — and writing must target the **destination** file. Both files are opened at once, one per mode:

Pseudocode:

```
OPENFILE "Old.txt" FOR READ
OPENFILE "New.txt" FOR WRITE
```

Python:

```
f_old = open("Old.txt", "r")
f_new = open("New.txt", "w")
```

Mark points: identifying the mode mismatch (2); correct separate opens (1).

Answer 34 (4 marks)
Pseudocode:

```
DECLARE LineOfText : STRING
OPENFILE "FileA.txt" FOR READ
OPENFILE "FileB.txt" FOR WRITE
READFILE "FileA.txt", LineOfText
WRITEFILE "FileB.txt", LineOfText
CLOSEFILE "FileA.txt"
CLOSEFILE "FileB.txt"
```

Python:

```
f_a = open("FileA.txt", "r")
f_b = open("FileB.txt", "w")
line_of_text = f_a.readline()
f_b.write(line_of_text)
f_a.close()
f_b.close()
```

Why: the folder roles are fixed — source for reading, destination for writing — and both drawers are shut afterwards.
Mark points: both opens in correct modes (1); read one line (1); write one line (1); both closed (1).

Section 7 — Standard algorithms and trace tables

Answer 35 (5 marks)
Pseudocode:

```
INPUT Value
Largest ← Value
Smallest ← Value
FOR I ← 2 TO 12
    INPUT Value
    IF Value > Largest
        THEN
            Largest ← Value
    ENDIF
    IF Value < Smallest
        THEN
            Smallest ← Value
    ENDIF
NEXT I
OUTPUT "Largest is ", Largest
OUTPUT "Smallest is ", Smallest
```

Python:

```
value = int(input("Number: "))
largest = smallest = value
for i in range(2, 13):
    value = int(input("Number: "))
    if value > largest:
        largest = value
    if value < smallest:
        smallest = value
print("Largest is", largest)
```

```
print("Smallest is", smallest)
```

Why: the first value seeds both records (better than 0 — negatives exist!), then 11 more values are compared one at a time.
Mark points: seeding both with the first value (1); loop covering the remaining 11 (1); update largest (1); update smallest (1); both outputs (1).

Answer 36 (2 marks)
`print(max(a))` is good Python but earns few or no marks here: the question asks for **the method**, and the syllabus's "standard methods of solution" include finding the maximum as an algorithm — the marker needs to see the seed-first-value-then-scan loop (or an equivalent step-by-step). Using `max(a)` shows awareness but skips the assessed logic. Save shortcuts for the scenario question, where clean Python is explicitly allowed.
Mark points: recognising the method itself is being tested (1); reference to the standard loop/seed-and-scan approach (1).

Answer 37 (4 marks)
T:
Iteration	I	A[I]	Found	Position
before loop	1	-	FALSE	0
1	1	7	FALSE	0
2	2	2	FALSE	0
3	3	9	TRUE	3
Output: 3.
Why: the finger scans lockers 1 and 2 (7, 2 — no match), finds 9 in locker 3 (`Found` becomes TRUE, `Position` = 3), and the loop condition `Found = FALSE` then fails — locker 4 is never visited. (The Python twin would print index 2.)
Mark points: three correct iterations (3); correct output 3 (1).

Answer 38 (3 marks)
Pseudocode:

```
INPUT Target
Found ← FALSE
FOR I ← 1 TO 5
    IF Names[I] = Target
        THEN
            Found ← TRUE
    ENDIF
NEXT I
IF Found = TRUE
    THEN
        OUTPUT "Yes"
    ELSE
        OUTPUT "No"
ENDIF
```

Python:

```
target = input("Name: ")
if target in names:          # the 'in' shortcut does the whole search
    print("Yes")
else:
    print("No")
```

Why: the loop is the register scan; Python's `in` runs exactly that scan behind the scenes. Case must match ("rahim" ≠ "Rahim") unless the question says otherwise.
Mark points: search loop over 5 names (1); correct flag handling (1); Yes/No output (1).

Answer 39 (5 marks)
T:
Pass	Comparison	Action	Array after step
1	5 vs 3	swap	3, 5, 4
1	5 vs 4	swap	3, 4, 5

| 2 | 3 vs 4 | no swap | 3, 4, 5 |
| 2 | 4 vs 5 | no swap | 3, 4, 5 |
Final array: `3, 4, 5`.
Why: pass 1 compares the two neighbour pairs, swapping both times (5 bubbles to the end). Pass 2 finds both pairs already in order — **no swaps at all**, which is the signal that the sort is finished.
Mark points: first swap correct (2); second swap (1); pass 2 with no swaps (1); final array and reasoning (1).

Answer 40 (3 marks)
When `Inner = N` the comparison reads `A[N + 1]` — outside the array (index out of range; Python IndexError). The inner loop must stop one element early:

```
    FOR Inner ← 1 TO N - Outer
```

(`TO N - 1` also acceptable.) The Python twin of the bug and fix:

```
for inner in range(0, n):          # WRONG on the last pass
for inner in range(0, n - 1 - outer):  # RIGHT
```

Mark points: identifying the out-of-range read (2); corrected bound (1).

Answer 41 (4 marks)
T:
Step	Value	Total	Count
before loop	-	0	0
1	5	5	1
2	-3	5	1
3	2	7	2
4	0	7	2
Output: `7, 2`.
Why: 5 passes the `> 0` test (till 5, click 1); −3 fails it (unchanged); 2 passes (till 7, click 2); 0 fails the test, is recorded in the row, and then `UNTIL Value = 0` stops the loop.
Mark points: `Total` column correct (2); `Count` column correct (1); correct output (1).

Answer 42 (2 marks)
A dry-run verifies the **logic** step by step without executing anything — the flight recorder shows every variable at every step, catching wrong conditions, uninitialised variables and arithmetic slips early (and on paper, without a computer). The same trace works for pseudocode and Python because the logic is what moves, not the syntax.
Mark points: checking/finding errors before running (1); tracking values at each step (1).

Section 8 — Program development and testing

Answer 43 (3 marks)
- **Presence check** — has any username been entered at all?
- **Length check** — is it exactly 6 characters (`LENGTH(Username) = 6` / `len(username) == 6`)?
- **Format check** — does it contain letters only (no digits or symbols)?
Mark points: one per check with a correct description.

Answer 44 (3 marks)
Validation (the bouncer) checks that input data is **reasonable and acceptable** — within range, right type/format, present — before processing. **Verification** (the proofreader) checks that data was **entered or transferred accurately** — e.g. a visual check against the source, or double entry where two typings are compared. One judges the data's sense; the other judges the accuracy of the entry.
Mark points: validation defined (1); verification defined (1); the difference made clear (1).

Answer 45 (4 marks)
(a) **Normal:** 15 (any sensible accepted age, 13–19).
(b) **Abnormal:** "abc" (text where a number is expected). Accept −5, 25, or any clearly invalid value.
(c) **Extreme:** 13 and 19 — the smallest and largest values that should be **accepted**.
(d) **Boundary:** 12, 13, 19, 20 — the extreme values **plus** the neighbouring values that must be

rejected.
Mark points: one per category; extreme and boundary need values on **both** sides.

Answer 46 (3 marks)
(a) **Analysis** — requirements and data identification happen here. (b) **Design** — structure diagrams are design tools. (c) **Testing** — running against sample data is the testing stage.
Mark points: one per task.

Section 9 — Flowcharts

Answer 47 (4 marks)
- **Terminator** (rounded box): "Start".
- **Input/Output** (parallelogram): "INPUT Password".
- **Decision** (diamond): "Password = 'Secret'?" with exits labelled **No** and **Yes**.
- **No** loops back along a flow line to the point above the input box.
- **Yes** leads to **Input/Output** (parallelogram): "OUTPUT 'Access granted'".
- **Terminator** (rounded box): "Stop".
Mark points: correct shapes for terminator, input/output and decision (2); No branch looping back (1); Yes branch with output and stop, labels present (1).

Answer 48 (4 marks)
- **Terminator**: "Start".
- **Process** box: "Total ← 0", "I ← 1".
- **Decision** diamond: "I <= 3?" — **Yes** exit down, **No** exit towards the output.
- Yes path: **Input/Output** "INPUT Value" → **Process** "Total ← Total + Value" → **Process** "I ← I + 1" → flow line back up to the decision.
- No path: **Input/Output** "OUTPUT Total" → **Terminator** "Stop".
Mark points: initialisation in a process box (1); decision with loop-back (1); body boxes in order (1); No path with output and stop (1).

Answer 49 (2 marks)
(a) A decision must be a **diamond** — the parallelogram is the Input/Output shape. (b) Both decision exits must be labelled (Yes/No or True/False) so the reader can follow each condition's road.
Mark points: one per error with its correction.

Section 10 — Boolean logic and logic gates

Answer 50 (3 marks)
T:
A	B	C	A OR B	NOT C	X
0	0	0	0	1	0
0	0	1	0	0	0
0	1	0	1	1	1
0	1	1	1	0	0
1	0	0	1	1	1
1	0	1	1	0	0
1	1	0	1	1	1
1	1	1	1	0	0
Output column: `0, 0, 1, 0, 1, 0, 1, 0`.
Why: build the two intermediate columns first; X takes 1 only where `A OR B` = 1 **and** `NOT C` = 1 (i.e. C = 0 and at least one of A, B is 1).
Mark points: correct intermediate columns (1); correct X column (2).

Answer 51 (2 marks)
OR lights for **one or both** inputs (1 OR 1 = 1) — either of two doors lets you in, even both. **XOR** lights only when the inputs **differ** (1 XOR 1 = 0) — one slice each, not both. They agree everywhere except the both-1 case.
Mark points: OR behaviour (1); XOR behaviour with the both-1 case (1).

Answer 52 (3 marks)
`X = (A AND NOT B) OR (B AND NOT A)` — the first term is "A is true and B is not", the second "B is true and A is not". This is the **XOR** gate, so `X = A XOR B` (or `A EOR B`) is equally correct.
Mark points: correct expanded expression (2); correct parentheses (1). Full marks for the XOR form.

Answer 53 (3 marks)
A **NOT** gate with input B. An **AND** gate whose two inputs are A (direct line) and the output of the NOT gate. The AND gate's output is X.
Mark points: AND gate (1); NOT gate on B (1); correct connections (1).

Answer 54 (4 marks)
T:

A	B	C	A AND B	NOT C	X
0	0	0	0	1	1
0	0	1	0	0	0
0	1	0	0	1	1
0	1	1	0	0	0
1	0	0	0	1	1
1	0	1	0	0	0
1	1	0	1	1	1
1	1	1	1	0	1

Output column: `1, 0, 1, 0, 1, 0, 1, 1`.
Why: X is 1 whenever `A AND B` is 1 **or** `NOT C` is 1. The only 0 rows are those with C = 1 while A and B are not both 1 (rows 001, 011, 101).
Mark points: one mark per pair of correct rows (4).

Section 11 — The scenario question

Answer 55 (10 marks)
Model answer — pseudocode:

```
FUNCTION GetMeals(Day : INTEGER) RETURNS INTEGER
    REPEAT
        OUTPUT "Meals sold on day ", Day
        INPUT Value
        IF Value < 0
            THEN
                OUTPUT "Invalid: the number must not be negative"
        ENDIF
    UNTIL Value >= 0
    RETURN Value
ENDFUNCTION
```

```
DECLARE Meals : ARRAY[1:10] OF INTEGER
DECLARE Day : INTEGER
FOR Day ← 1 TO 10
Meals[Day] ← GetMeals(Day)
NEXT Day

Total ← 0
FOR Day ← 1 TO 10
Total ← Total + Meals[Day]
NEXT Day
Average ← Total / 10
OUTPUT "Total meals sold: ", Total
OUTPUT "Average per day: ", Average

Highest ← Meals[1]
FOR Day ← 2 TO 10
IF Meals[Day] > Highest
THEN
Highest ← Meals[Day]
```

```
ENDIF
NEXT Day
OUTPUT "Highest day: ", Highest

Above ← 0
FOR Day ← 1 TO 10
IF Meals[Day] > Average
THEN
Above ← Above + 1
ENDIF
NEXT Day
OUTPUT "Days above average: ", Above
```

Model answer — Python (shortcuts used):

```
def get_meals(day):
"""Ask for one day's sales, rejecting negative values."""
while True:
value = int(input(f"Meals sold on day {day}: "))
if value >= 0:
return value
print("Invalid: the number must not be negative")

def find_highest(values):
highest = values[0]
for v in values:
if v > highest:
highest = v
return highest

meals = []
for day in range(1, 11):
meals.append(get_meals(day))

total = 0
for v in meals:
total += v
average = total / 10
print(f"Total meals sold: {total}")
print(f"Average per day: {average}")

print(f"Highest day: {find_highest(meals)}")

above = 0
for v in meals:
if v > average:
above += 1
print(f"Days above average: {above}")
```

Why: the validation uses REPEAT ... UNTIL (at least one attempt, stops when valid — the tea loop); the ten
Mark points (10, identical for either language): loop collecting exactly 10 stored values (2); validation rejec

Appendix A — pseudocode and Python side-by-side reference

Pseudocode	Python
DECLARE X : INTEGER	x = 0 (comment the type)
CONSTANT Rate ← 5	RATE = 5
X ← expr	x = expr
INPUT X	x = int(input("..."))
OUTPUT A, "is", B	print(A, "is", B) or f-string
IF c THEN ... ELSE ... ENDIF	if c: ... else: ...
nested IF inside ELSE	nested if in else (or elif ladder)
CASE OF X ... ENDCASE	nested if / elif ladder
FOR I ← 1 TO N ... NEXT I	for i in range(1, N + 1):
FOR I ← A TO B STEP S	for i in range(A, B + 1, S):
loop over elements	for value in a: (for-each)
WHILE c DO ... ENDWHILE	while c:
REPEAT ... UNTIL c	while True: ... if c: break
Total ← Total + Value	total += value (also -=, *=, /=)
swap via Temp	a[i], a[j] = a[j], a[i]
IF Ch = "A" OR Ch = "E" OR ...	if ch in "AEIOU":
manual total / max / min	sum(a) / max(a) / min(a)
OUTPUT repeated characters	print("-" * 20)
OUTPUT "Total: ", T	print(f"Total: {T}")
reverse loop with STEP -1	s[::-1]
DIV(A, B)	A // B
MOD(A, B)	A % B
A ^ B	A ** B
X <> Y	X != Y
ARRAY[1:N] OF INTEGER	a = [0] * N (indexes 0 to N-1)
Grid[2, 3]	grid[1][2]
LENGTH(s)	len(s)
SUBSTRING(s, p, l)	s[p-1 : p-1+l]
UCASE(s) / LCASE(s)	s.upper() / s.lower()
ROUND(x, n) / RANDOM()	round(x, n) / random.random()
PROCEDURE P(x : INTEGER) ... CALL P(5)	def p(x): ... p(5)
FUNCTION F(x) RETURNS INTEGER ...	def f(x): return ...
// comment	# comment
OPENFILE "F" FOR READ / WRITE	open("F", "r") / open("F", "w")
READFILE "F", v	v = f.readline()
WRITEFILE "F", v	f.write(str(v))
CLOSEFILE "F"	f.close()

Appendix B — command words

Command word	What it means
Calculate	work out from given facts, figures or information
Compare	identify similarities and/or differences
Define	give a precise meaning
Demonstrate	show how, or give an example
Describe	state the points / give characteristics and main features
Evaluate	judge quality, importance, amount or value
Explain	give purposes or reasons; make relationships clear; say why/how
Give	produce an answer from a source or from memory
Identify	name, select or recognise
Outline	set out the main points
Show (that)	provide structured evidence leading to a given result
State	express in clear terms
Suggest	apply knowledge to a situation with a range of valid responses